

Ngăn xếp, hàng đợi và heap (Stacks, Queues, Heaps)

PHỤ THUỘC B/06 cho kỹ thuật quét tuyến tính; A/03 cho phân tích khẩu hao — ngăn xếp đơn điệu là ví dụ khẩu hao đẹp nhất trong cả quyển sách.

TÓM TẮT

Ba cấu trúc trong chương này trả lời cùng một câu hỏi với ba câu trả lời khác nhau: *trong đồng dư liệu đang chờ, phần tử nào nên xử lý tiếp?* Ngăn xếp nói “cái mới nhất”, hàng đợi nói “cái cũ nhất”, heap nói “cái nhỏ nhất”. Ba lựa chọn ấy sinh ra ba họ thuật toán rất khác nhau: đệ quy và duyệt sâu, duyệt rộng và mô phỏng theo thời gian, và các thuật toán tham lam kiểu Dijkstra hay Huffman. Chương cũng dựng hai kỹ thuật có tỉ lệ sức mạnh trên độ dài code rất cao: ngăn xếp đơn điệu và hàng đợi hai đầu đơn điệu. Nếu chỉ nhớ một điều: heap cổ tình duy trì *ít trật tự hơn* mảng đã sắp — nó chỉ biết ai nhỏ nhất — và chính sự tiết chế đó làm nó rẻ.

KIẾN THỨC CẦN CÓ TRƯỚC

- **ngăn xếp (stack)** — cấu trúc vào sau ra trước (LIFO): thêm và lấy đều ở một đầu, cả hai $O(1)$.
- **hàng đợi (queue)** — vào trước ra trước (FIFO): thêm ở một đầu, lấy ở đầu kia.
- **hàng đợi hai đầu (deque)** — thêm và lấy được ở cả hai đầu, mỗi thao tác $O(1)$.
- **heap nhị phân** — cây nhị phân gần đầy, mỗi nút không lớn hơn các con của nó, lưu bằng mảng: con của i là $2i$ và $2i + 1$. Lấy nhỏ nhất $O(1)$, thêm và xóa $O(\log n)$.
- **hàng đợi ưu tiên (priority queue)** — giao diện trừu tượng “thêm phần tử, lấy phần tử ưu tiên nhất”; heap là cách cài phổ biến nhất nhưng không phải duy nhất.
- **ngăn xếp đơn điệu** — ngăn xếp mà các phần tử trong nó luôn giữ thứ tự tăng (hoặc giảm); dùng để tìm “phần tử lớn hơn kế tiếp” cho mọi vị trí trong $O(n)$.

CÂU HỎI MỞ ĐẦU

- Q1. Ngăn xếp đơn điệu có vòng lặp lồng nhau và trông như $O(n^2)$. Vì sao nó thật sự là $O(n)$?
- Q2. Vì sao heap chỉ cho biết phần tử nhỏ nhất mà không cho biết phần tử thứ hai một cách trực tiếp — và vì sao đó lại là *ưu điểm* chứ không phải khuyết điểm?
- Q3. Xây heap từ n phần tử tốn $O(n)$ chứ không phải $O(n \log n)$, dù mỗi lần chèn tốn $O(\log n)$. Chỗ nào trong lập luận “ n lần chèn” bị sai?
- Q4. Duyệt sâu dùng ngăn xếp, duyệt rộng dùng hàng đợi — chỉ khác một dòng code. Vì sao hai thuật toán lại có tính chất khác nhau đến vậy?
- Q5. Khi nào một mảng đã sắp xếp tốt hơn heap, và ngược lại?

Chương trước quét dữ liệu theo thứ tự chỉ số. Nhưng rất nhiều bài toán không cho phép làm vậy: bạn xử lý một phần tử, việc đó sinh ra công việc mới, và công việc mới phải xếp vào một chỗ nào đó để xử lý sau. Câu hỏi trung tâm trở thành: *trong đồng công việc đang chờ, cái nào làm tiếp?*

Ba cấu trúc trong chương này là ba câu trả lời, và điều đáng chú ý là chúng chỉ khác nhau ở đúng chỗ đó — cùng một khung “lấy ra, xử lý, đẩy vào” nhưng đổi quy tắc lấy ra thì được ba thuật toán khác hẳn nhau về hành vi. Duyệt sâu, duyệt rộng và Dijkstra là cùng một chương trình với ba cấu trúc chờ khác nhau, và nhận ra điều đó làm ba thuật toán ấy sụp thành một.

1.1 Ngăn xếp: cái mới nhất trước

Ngăn xếp là cấu trúc gắn liền với *sự lồng nhau*. Mỗi khi bài toán có cấu trúc “mở rồi phải đóng theo đúng thứ tự ngược lại” thì ngăn xếp là công cụ tự nhiên: dấu ngoặc trong biểu thức, thẻ trong tài liệu đánh dấu, lời gọi hàm trong chương trình, thao tác hoàn tác trong trình soạn thảo.

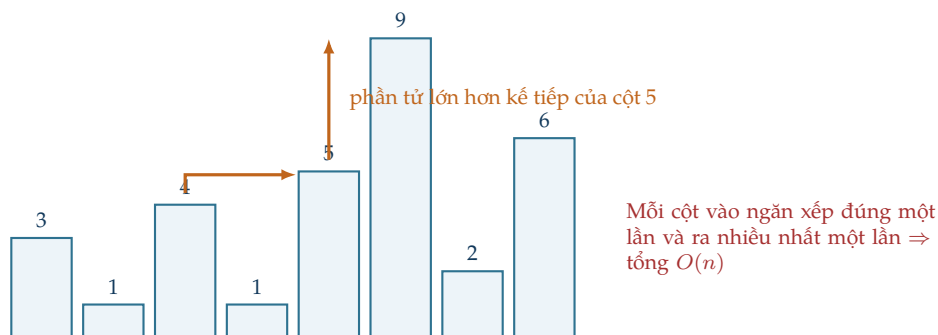
Điều đáng để ý về mặt khái niệm: đệ quy và ngăn xếp là cùng một thứ. Máy tính thực hiện đệ quy bằng cách đẩy trạng thái lời gọi vào ngăn xếp phân cứng. Vì vậy mọi thuật toán đệ quy đều viết lại được thành vòng lặp với ngăn xếp tường minh — một kỹ thuật đáng biết vì đôi khi nó là cách duy nhất tránh tràn ngăn xếp khi độ sâu lên tới hàng trăm nghìn (E/25).

Ngăn xếp đơn điệu: kỹ thuật đáng giá nhất của chương

Bài toán: với mỗi vị trí i trong mảng, tìm vị trí đầu tiên bên phải có giá trị lớn hơn a_i . Vết cạn $O(n^2)$. Ngăn xếp đơn điệu cho $O(n)$.

Ý tưởng: quét từ trái sang phải, duy trì một ngăn xếp chứa các vị trí mà *chưa tìm được phần tử lớn hơn*. Khi tới vị trí i , lấy ra khỏi ngăn xếp mọi vị trí có giá trị nhỏ hơn a_i — với mỗi vị trí lấy ra, đáp án của nó chính là i . Rồi đẩy i vào.

Bất biến: *ngăn xếp luôn chứa các giá trị giảm dần từ đáy lên đỉnh, và đó đúng là những vị trí còn đang chờ đáp án*. Vì sao $O(n)$? Mỗi vị trí được đẩy vào đúng một lần và lấy ra nhiều nhất một lần, nên tổng số thao tác không quá $2n$ — lập luận khấu hao y hệt lập luận “tổng quãng đường con trỏ” ở chương B/06. Vòng lặp trong có thể chạy nhiều bước ở một số lần lặp, nhưng khi đó nó vừa xoá đi nhiều phần tử mà ai đó đã trả tiền lúc đẩy vào.



Hình 1.1. Ngăn xếp đơn điệu tìm “phần tử lớn hơn kế tiếp”. Các cột đang nằm trong ngăn xếp luôn giảm dần từ trái sang phải; khi gặp một cột cao hơn, tất cả các cột thấp hơn nó bị lấy ra và nhận cột này làm đáp án. Cùng cấu trúc giải bài “hình chữ nhật lớn nhất trong biểu đồ cột” — ở đó mỗi cột bị lấy ra chính là lúc ta biết được cả biên trái lẫn biên phải của hình chữ nhật cao bằng nó.

Họ bài giải bằng ngăn xếp đơn điệu rộng hơn tương ứng: phần tử lớn hơn/nhỏ hơn kế tiếp hai phía, hình chữ nhật lớn nhất trong biểu đồ cột, nước mưa mắc kẹt, số đoạn con mà một phần tử là cực đại, và cả việc dựng cây Cartesian. Dấu hiệu nhận biết trên đề: *với mỗi phần tử, cần biết phần tử gần nhất về một phía thoả một quan hệ so sánh*.

1.2 Hàng đợi và hàng đợi hai đầu

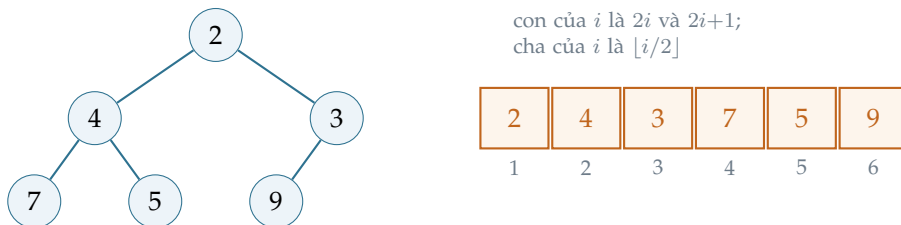
Hàng đợi gắn với việc xử lý theo *lớp*: duyệt rộng trên đồ thị xử lý hết các đỉnh cách 1 bước rồi mới tới các đỉnh cách 2 bước, và chính tính chất đó làm duyệt rộng cho đường đi ngắn nhất trên đồ thị không trọng số (C/17). Trong hệ thống thật, hàng đợi là cấu trúc của mọi thứ có “xếp hàng chờ”: hàng đợi tin nhắn, bộ đệm, lịch tiến trình.

Hàng đợi hai đầu cho một kỹ thuật quan trọng: *cực đại trên cửa sổ trượt* trong $O(n)$. Duy trì một deque chứa các chỉ số có giá trị giảm dần; khi cửa sổ tiến, bỏ ở đầu những chỉ số đã rơi ra ngoài cửa sổ, và bỏ ở cuối những chỉ số có giá trị nhỏ hơn phần tử mới — vì chúng không bao giờ còn cơ hội làm cực đại nữa. Đây là kết hợp của cửa sổ trượt (B/06) với ý tưởng đơn điệu của mục trước, và nó cũng là bước tăng tốc chuẩn cho một họ bài quy hoạch động (C/16).

1.3 Heap: chỉ biết ai nhỏ nhất, và chỉ cần thế

Heap nhị phân là một cây nhị phân gần đầy lưu trong mảng, với tính chất: mỗi nút không lớn hơn các con của nó. Chú ý điều *không* được đảm bảo: hai anh em không có quan hệ gì với nhau, và mảng biểu diễn heap hoàn toàn không sắp xếp. Heap chỉ biết đúng một điều — gốc là nhỏ nhất.

Sự tiết chế đó chính là nguồn sức mạnh. Duy trì trật tự toàn phần (mảng đã sắp) tốn $O(n)$ cho mỗi lần chèn; duy trì trật tự bộ phận của heap chỉ tốn $O(\log n)$. Nếu bài toán chỉ cần biết ai nhỏ nhất — và rất nhiều bài chỉ cần thế — thì trả tiền cho trật tự toàn phần là lãng phí.



Hình 1.2. Heap nhị phân và biểu diễn mảng của nó. Không có con trỏ nào — quan hệ cha con là phép tính chỉ số, nên heap vừa tiết kiệm bộ nhớ vừa thân thiện với cache. Chú ý mảng *không* sắp xếp: 4 đứng trước 3. Heap chỉ hứa mỗi nút nhỏ hơn các con của nó, không hứa gì về quan hệ giữa các nhánh.

Ba thao tác và chi phí: lấy nhỏ nhất $O(1)$; thêm phần tử $O(\log n)$ bằng cách đặt vào cuối rồi đẩy lên; xoá nhỏ nhất $O(\log n)$ bằng cách đưa phần tử cuối lên gốc rồi đẩy xuống. Còn một thao tác đáng nhớ vì nó phản trực giác: *xây heap từ n phần tử có sẵn tốn $O(n)$, không phải $O(n \log n)$* . Lý do: nếu đi từ dưới lên và đẩy xuống, thì các nút ở tầng đáy — chiếm một nửa số nút — gần như không phải làm gì, các nút ở tầng trên thì ít. Tổng chi phí là $\sum_h (n/2^h) \cdot h$, một chuỗi hội tụ về $O(n)$. Đây là ứng dụng trực tiếp của “cấp số nhân giảm bị chặn bởi hằng số” ở chương A/05.

Ba mẫu dùng heap đáng thuộc

Chọn k phần tử lớn nhất trong n phần tử. Giữ một heap nhỏ nhất kích thước k ; mỗi phần tử mới, nếu lớn hơn gốc thì thay gốc. Chi phí $O(n \log k)$ và bộ nhớ $O(k)$ — tốt hơn hẳn việc sắp toàn bộ khi $k \ll n$, và quan trọng hơn, nó chạy được cả khi dữ liệu là luồng không lưu hết được (D/24).

Trộn k danh sách đã sắp. Heap chứa phần tử đầu của mỗi danh sách; mỗi lần lấy ra cái nhỏ nhất rồi nạp phần tử kế tiếp của danh sách đó. Chi phí $O(N \log k)$. Đây là thuật toán nền của việc sắp xếp ngoài (dữ liệu lớn hơn bộ nhớ) và của phép trộn trong nhiều hệ cơ sở dữ liệu.

Trung vị của luồng dữ liệu. Hai heap: một heap lớn nhất giữ nửa nhỏ, một heap nhỏ nhất giữ nửa lớn, giữ cho kích thước chênh nhau tối đa 1. Trung vị luôn ở đỉnh của một trong hai. Mẫu này — *tách dữ liệu thành hai nửa, mỗi nửa một heap quay ngược nhau* — còn dùng cho nhiều bài “duy trì phân vị”.

Cấu trúc	Lấy nhỏ nhất	Chèn	Tìm phần tử thứ k	Nên chọn khi
Mảng chưa sắp	$O(n)$	$O(1)$	$O(n)$	Dữ liệu tĩnh, ít truy vấn
Mảng đã sắp	$O(1)$	$O(n)$	$O(1)$	Xây một lần, truy vấn nhiều, không chèn
Heap nhị phân	$O(1)$	$O(\log n)$	<i>không hỗ trợ</i>	Chỉ cần cực trị, có chèn liên tục
Cây cân bằng (B/09)	$O(\log n)$	$O(\log n)$	$O(\log n)$	Cần cả thứ tự lẫn cực trị

Bảng trên trả lời câu hỏi mở đầu thứ năm: heap thắng khi có chèn liên tục và chỉ cần cực trị; mảng đã sắp thắng khi dữ liệu tĩnh và cần truy vấn theo hạng; cây cân bằng đắt hơn cả hai nhưng làm được cả hai việc.

CẠM BẤY

Heap nhị phân chuẩn *không* hỗ trợ hai thao tác mà người ta hay tưởng là có: xoá một phần tử bất kỳ, và giảm khoá của một phần tử. Muốn có chúng cần lưu thêm chỉ số vị trí của từng phần tử, tức là một heap có đánh dấu. Trong lập trình thi đấu, mẹo thông dụng hơn là *xoá lười*: cứ đẩy bản ghi mới vào và khi lấy ra thì bỏ qua những bản ghi đã lỗi thời. Đây chính là lý do bản Dijkstra viết bằng hàng đợi ưu tiên thường có dòng kiểm tra “nếu khoảng cách lấy ra lớn hơn khoảng cách đã biết thì bỏ qua” (C/17).

1.4 Cùng một khung, ba cấu trúc, ba thuật toán

Đây là chỗ chương này trả công nhiều nhất về mặt tư duy. Xét khung chương trình sau: lấy một phần tử từ kho chờ, xử lý nó, đẩy các phần tử liên quan vào kho chờ, lặp lại. Đổi kho chờ, bạn được ba thuật toán khác nhau:

- **Ngăn xếp** → duyệt sâu. Đi tới cùng một nhánh rồi mới quay lại. Dùng để tìm thành phần liên thông, phát hiện chu trình, sắp thứ tự tô-pô.
- **Hàng đợi** → duyệt rộng. Xử lý theo lớp khoảng cách. Cho đường đi ngắn nhất trên đồ thị không trọng số.
- **Hàng đợi ưu tiên** → Dijkstra và A*. Xử lý theo thứ tự chi phí tăng dần. Cho đường đi ngắn nhất với trọng số không âm.

Nhận ra ba thuật toán này là một giúp bạn nhớ ít hơn và suy được nhiều hơn. Ví dụ, câu hỏi “vì sao Dijkstra cần trọng số không âm” trở thành hiển nhiên khi nhìn theo khung này: thuật toán chốt một đỉnh ngay khi lấy nó ra khỏi hàng đợi ưu tiên, tức là nó tin rằng không có đường nào rẻ hơn sẽ tới sau — và niềm tin ấy chỉ đúng nếu mọi cạnh đều làm chi phí tăng lên. Chương C/17 khai thác đầy đủ cách nhìn này.

1.5 Giảm khoá và bài học Fibonacci heap: khi cận và máy thật không cùng kết luận

Khối *Cạm bẫy* ở trên nói heap nhị phân không có thao tác giảm khoá. Câu hỏi tự nhiên là: có cấu trúc nào có không, và nó đáng giá bao nhiêu? Câu trả lời dẫn tới một trong những câu chuyện đáng học nhất của ngành cấu trúc dữ liệu.

Bắt đầu từ chỗ thao tác này thực sự tốn tiền. Dijkstra (C/17) mỗi lần thư giãn một cạnh là một lần muốn hạ nhãn của một đỉnh. Với heap nhị phân, cách làm phổ biến là đẩy thêm một bản ghi

mới rồi bỏ qua bản ghi lỗi thời khi lấy ra — nghĩa là hàng đợi chứa tới m phần tử và tổng chi phí là $O(m \log n)$. Fredman và Tarjan[1] đặt câu hỏi: nếu giảm khoá rẽ đi thì Dijkstra rẽ đi bao nhiêu? Với $O(1)$ cho giảm khoá và $O(\log n)$ cho lấy nhỏ nhất, tổng thành $O(m + n \log n)$ — tốt hơn hẳn khi đồ thị dày, vì lúc đó m lớn hơn $n \log n$ nhiều lần.

Fibonacci heap đạt hai cận đó theo nghĩa *khấu hao*, và cơ chế của nó là một minh hoạ đẹp cho phương pháp thể năng ở chương A/03: hoãn mọi việc dọn dẹp tới lúc bắt buộc. Chèn chỉ là thêm một cây vào danh sách gốc, không so sánh gì cả. Giảm khoá chỉ là *cắt* nút đó ra khỏi cha và biến nó thành một gốc mới — cũng không so sánh gì. Toàn bộ việc gộp các cây cùng bậc bị dồn sang lần lấy nhỏ nhất kế tiếp, và để cây không bị cắt tới mức xơ xác, mỗi nút bị mất con lần thứ hai sẽ bị cắt lên theo dây chuyền. Hàm thể năng đếm số cây gốc cùng số nút đã bị đánh dấu, và chính nó trả tiền cho những lần dọn dẹp đất đỏ.

Đến đây lý thuyết rất đẹp. Nhưng câu hỏi thực dụng — *có nên dùng không* — lại được trả lời bằng máy đo chứ không bằng chứng minh, và câu trả lời không đơn giản như hai phe vẫn nói. Larkin, Sen và Tarjan[2] cài mười biến thể heap theo cùng một chuẩn “không tối ưu hoá lắt léo” rồi đo trên ba loại tải: dãy thao tác nhân tạo, Dijkstra trên bản đồ đường thật, và một bộ mô phỏng sự kiện rời rạc. Bốn kết luận của họ đáng nhớ hơn bất kỳ bảng độ phức tạp nào.

Thứ nhất, thời gian chạy tương quan mạnh với *tỉ lệ trượt cache*, đặc biệt là cache L1 — nghĩa là quyết định thiết kế ở tầng cao nhất, “mảng hay con trỏ”, chi phối kết quả nhiều hơn cận khấu hao. Thứ hai, pairing heap thắng Fibonacci heap trên mọi tải mà họ thử: cùng ý tưởng hoãn dọn dẹp nhưng ít siêu dữ liệu hơn nên ít trượt cache hơn. Thứ ba, cả hai điều mà “ai cũng biết” đều sai ở một số tải: heap d phân lưu trong mảng *không* phải luôn tốt nhất, và Fibonacci heap *không* phải luôn chậm thảm hại — có tải nó vượt heap mảng. Thứ tư, và mỉa mai nhất, những biến thể được đề xuất về sau nhằm *đơn giản hoá* Fibonacci heap thường chạy chậm hơn chính bản gốc.

Bài học rút ra không phải “đừng tin lý thuyết” mà là một câu chính xác hơn: cận khấu hao đếm *số thao tác trên cấu trúc*, còn máy tính tiền theo *cách bố trí bộ nhớ*. Hai thước đo ấy chỉ trùng nhau khi mỗi thao tác có giá như nhau, mà điều đó không đúng từ khi có phân cấp cache (A/03). Vì vậy quy tắc thực hành gọn lại thành ba dòng: trong thi đấu, dùng heap nhị phân của thư viện kèm mẹo xoá lười; trong hệ thống thật mà hàng đợi ưu tiên nằm trên đường nóng, hãy đo với đúng tải của mình (E/26); và chỉ nghĩ tới Fibonacci heap khi đồ thị dày tới mức số lần giảm khoá lớn hơn hẳn số lần lấy nhỏ nhất — một điều kiện hiếm gặp hơn nhiều so với ấn tượng mà giáo trình để lại.

1.6 Gốc rễ: từ máy tính không có đệ quy tới lịch trình hệ điều hành

Ngăn xếp có một lịch sử gắn liền với việc làm cho đệ quy trở nên khả thi. Những máy tính đầu tiên không có cơ chế gọi hàm lồng nhau: địa chỉ trở về được ghi đè vào một ô cố định, nên hàm không thể gọi lại chính nó. Cuối thập niên 1950, khi ngành trình biên dịch cần đánh giá biểu thức toán học có dấu ngoặc lồng nhau, hai nhà nghiên cứu người Đức là Friedrich Bauer và Klaus Samelson đề xuất dùng một cấu trúc vào sau ra trước để lưu các phép toán đang chờ — và cùng thời gian đó, việc thiết kế ngôn ngữ Algol 60 với đệ quy đã biến ngăn xếp thành một phần của kiến trúc máy. Ngày nay “ngăn xếp lời gọi” là khái niệm mà mọi lập trình viên gặp mỗi lần đọc thông báo lỗi, nhưng nó là một phát minh có ngày tháng, không phải điều hiển nhiên.

Heap ra đời năm 1964 từ một nhu cầu rất cụ thể: J. W. J. Williams muốn một thuật toán sắp xếp tại chỗ với đảm bảo $O(n \log n)$ ở trường hợp xấu nhất — thứ mà quicksort không có — và ông thiết kế cấu trúc heap để phục vụ heapsort. Ngay sau đó, Robert Floyd đưa ra cách xây heap từ dưới lên trong $O(n)$. Điều thú vị là cấu trúc sinh ra để phục vụ sắp xếp lại trở nên nổi tiếng hơn ở vai trò khác: hàng đợi ưu tiên cho mô phỏng sự kiện rời rạc, cho lịch trình hệ điều hành, và cho các thuật toán tham lam trên đồ thị.

Câu chuyện đó lặp lại một lần nữa hai mươi năm sau. Fredman và Tarjan thiết kế Fibonacci heap [1] với mục tiêu rất hẹp: giảm độ phức tạp của Dijkstra từ $O(m \log n)$ xuống $O(m + n \log n)$, bằng cách làm cho thao tác giảm khoá tốn $O(1)$ khẩu hao. Họ đạt được mục tiêu lý thuyết, nhưng hằng số lớn tới mức trong thực tế heap nhị phân thường vẫn nhanh hơn — một ví dụ kinh điển cho bài học “tiệm cận không phải tất cả” ở chương A/03, và cũng là ví dụ cho thấy một kết quả lý thuyết vẫn có giá trị dù ít khi được cài đặt: nó xác lập đâu là giới hạn.

Gốc rễ: ý tưởng này ra đời từ bài toán nào

Mốc	Bài toán thúc ép lúc đó	Điều thay đổi sau đó
Cuối thập niên 1950 — Bauer, Samelson và Algol 60	Đánh giá biểu thức có ngoặc lồng nhau; và làm cho hàm gọi được chính nó.	Ngăn xếp trở thành cấu trúc nền của trình biên dịch và của kiến trúc máy; “ngăn xếp lời gọi” ra đời.
1964 — Williams	Cần thuật toán sắp xếp tại chỗ có đảm bảo $O(n \log n)$ ở trường hợp xấu nhất.	Heap nhị phân và heapsort; cấu trúc sau đó nổi tiếng hơn ở vai trò hàng đợi ưu tiên.
1964 — Floyd	Xây heap bằng n lần chèn tốn $O(n \log n)$, có cách rẻ hơn không?	Xây từ dưới lên trong $O(n)$; ví dụ mẫu mực cho việc tổng của cấp số nhân giảm là hằng số.
Thập niên 1960–70 — mô phỏng sự kiện rời rạc, hệ điều hành	Cần lấy sự kiện sắp xảy ra sớm nhất trong hàng triệu sự kiện đang chờ.	Hàng đợi ưu tiên trở thành cấu trúc lõi của mô phỏng và của bộ lập lịch.
1984–87 — Fredman & Tarjan	Dijkstra bị chặn bởi chi phí giảm khoá.	Fibonacci heap: giảm khoá $O(1)$ khẩu hao, cho $O(m + n \log n)$; nhưng hằng số lớn nên hiếm khi thắng trong thực tế.

1.7 Liên hệ ngang

Liên hệ ngang: ý này còn sống ở đâu nữa

Lĩnh vực	Ý tưởng tương ứng	Giống ở đâu, khác ở đâu
Kiến trúc máy tính	Ngăn xếp lời gọi, khung ngăn xếp, tràn ngăn xếp	Không phải chỉ tương tự — đệ quy <i>được cài đặt</i> bằng đúng cấu trúc này. Hệ quả thực dụng: đệ quy sâu quá thì sập, và cách chữa là viết lại bằng ngăn xếp tường minh (E/25).
Trình biên dịch	Phân tích biểu thức, khớp ngoặc, ký pháp Ba Lan ngược	Ứng dụng lịch sử đầu tiên của ngăn xếp. Mọi bài “kiểm tra dãy ngoặc hợp lệ” trong phòng vẫn là phiên bản thu nhỏ của bộ phân tích cú pháp thật.
Hệ điều hành	Lịch trình tiến trình theo độ ưu tiên, hàng đợi sẵn sàng	Cùng cấu trúc hàng đợi ưu tiên. Khác ở ràng buộc: bộ lập lịch phải tránh bỏ sót tiến trình, nên độ ưu tiên thay đổi theo thời gian chờ — một yêu cầu không có trong bài toán thuật toán thuần túy.
Mô phỏng sự kiện rời rạc	Lịch sự kiện: luôn lấy sự kiện có thời điểm sớm nhất	Đây là ứng dụng “đúng nghĩa đen” của hàng đợi ưu tiên, và là lý do cấu trúc này quan trọng ngoài phạm vi thuật toán đồ thị.
Lập kế hoạch đường đi cho robot	A^* với biên là hàng đợi ưu tiên	Cùng khung “lấy ra, mở rộng, đẩy vào” ở mục 4, chỉ đổi khoá ưu tiên thành $g + h$. Bài học chuyển thẳng: chọn cấu trúc kho chờ chính là chọn thuật toán.
Xử lý dòng dữ liệu	Giữ k phần tử lớn nhất bằng heap kích thước k	Cùng mẫu ở mục 3.1. Ưu điểm quyết định trong bối cảnh luồng: bộ nhớ $O(k)$ không phụ thuộc độ dài luồng (D/24).

Lĩnh vực	Ý tưởng tương ứng	Giống ở đâu, khác ở đâu (tiếp)
Nén dữ liệu	Thuật toán Huffman: luôn gộp hai tần suất nhỏ nhất	Một thuật toán tham lam mà hàng đợi ưu tiên là công cụ thực thi trực tiếp. Xem C/15 cho phần chứng minh vì sao tham lam ở đây đúng.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Vì mỗi phần tử được đẩy vào ngăn xếp đúng một lần và bị lấy ra nhiều nhất một lần trong suốt cả thuật toán, nên tổng số thao tác không quá $2n$ bất kể vòng lặp trong chạy dài tới đâu ở một số lần lặp cụ thể. Đây là lập luận khấu hao kiểu tổng gộp ($A/03$): một lần lấy ra nhiều phần tử chỉ xảy ra được sau khi đã có nhiều lần đẩy vào tương ứng. Cùng dạng lập luận với “tổng quãng đường con trỏ” ở B/06.
- A2. Vì duy trì ít trật tự hơn thì rẻ hơn. Sắp xếp toàn phần đòi $O(n \log n)$ và mỗi lần chèn vào mảng đã sắp tồn $O(n)$; heap chỉ duy trì quan hệ cha-con nên chèn chỉ $O(\log n)$. Nếu bài toán chỉ hỏi “ai nhỏ nhất” — và rất nhiều thuật toán tham lam chỉ hỏi đúng thế — thì trả tiền cho trật tự toàn phần là lãng phí. Đây là ví dụ tiêu biểu của nguyên tắc chung ở phần B: chọn cấu trúc duy trì *đúng* lượng trật tự mà bài cần, không hơn.
- A3. Sai ở chỗ giả định mọi phần tử đều phải đi hết $\log n$ tầng. Khi xây từ dưới lên, một nửa số nút nằm ở tầng đáy và không phải đẩy xuống bước nào; một phần tư ở tầng kế và đi tối đa một bước; cứ thế. Tổng chi phí là $\sum_h (n/2^h) \cdot h$, một chuỗi hội tụ, cho $O(n)$. Còn cách chèn lần lượt thì ngược lại: phần tử vào sau đi từ đáy lên nên đúng là tốn $O(\log n)$ mỗi cái.
- A4. Vì thứ tự lấy ra quyết định thứ tự khám phá. Ngăn xếp lấy cái mới nhất nên thuật toán lao sâu theo một nhánh trước khi quay lại — hợp cho việc phát hiện chu trình và tính thứ tự tô-pô, vì thời điểm vào/ra của mỗi đỉnh mang thông tin cấu trúc. Hàng đợi lấy cái cũ nhất nên khám phá theo lớp khoảng cách tăng dần — đó chính xác là lý do duyệt rộng cho đường đi ngắn nhất trên đồ thị không trọng số. Cùng chương trình, đổi một dòng, đổi hẳn tính chất được đảm bảo.
- A5. Mảng đã sắp thẳng khi dữ liệu tĩnh: nó cho cả cực trị lẫn truy vấn phần tử thứ k trong $O(1)$, còn heap không trả lời được câu thứ hai. Heap thẳng khi có chèn xen kẽ liên tục, vì chèn vào mảng đã sắp tồn $O(n)$ còn heap chỉ $O(\log n)$; và heap còn thẳng khi bạn chỉ cần k phần tử đầu của một luồng rất dài, vì bộ nhớ chỉ $O(k)$. Cần cả hai khả năng thì phải trả giá cao hơn bằng cây cân bằng ở B/09.

NÓI LẠI THẬT ĐƠN GIẢN

Ba cấu trúc này chỉ khác nhau ở một câu hỏi: trong đồng việc đang chờ, làm việc nào trước? Ngăn xếp giống chồng giấy tờ trên bàn: bạn luôn xử lý tờ trên cùng, tức là tờ mới đặt xuống gần nhất. Kiểu này hợp khi công việc lồng nhau — đang làm việc A thì phát sinh việc B phải xong trước, xong B mới quay lại A. Đó cũng đúng là cách máy tính chạy hàm gọi hàm.

Hàng đợi giống hàng người chờ mua vé: ai tới trước phục vụ trước. Kiểu này hợp khi bạn muốn lan toả đều ra xung quanh, ví dụ “tìm những chỗ cách đây đúng ba bước” — vì mọi chỗ cách hai bước sẽ được xử lý xong trước khi chạm tới chỗ cách ba bước.

Heap giống phòng cấp cứu: không xét ai tới trước, mà xét ai nặng nhất. Điều đáng nói là phòng cấp cứu không cần xếp hạng toàn bộ bệnh nhân từ nặng tới nhẹ; nó chỉ cần biết *ai nặng nhất ngay lúc này*. Chính vì đòi hỏi ít hơn nên nó làm nhanh hơn — và đó là bài học chung: đừng sắp xếp mọi thứ khi bạn chỉ cần biết cái đứng đầu.

Còn ngăn xếp đơn điệu thì giống việc bạn xếp các cột chờ “ai sẽ cao hơn tôi”. Mỗi cột mới cao lên sẽ giải quyết xong một loạt cột thấp đang chờ, và những cột đó rời hàng vĩnh viễn. Vì mỗi cột vào hàng một lần và rời hàng một lần, tổng công việc chỉ tỉ lệ với số cột, dù có lúc một cột mới giải quyết hết cả chục cột cũ.

Một câu: Chọn cấu trúc kho chờ chính là chọn thuật toán: mới nhất, cũ nhất, hay nhỏ nhất.

Chỗ không rút gọn được: Việc heap không hỗ trợ “xoá phần tử bất kỳ” là hạn chế thật, không phải chi tiết cài đặt; muốn có nó thì phải trả bằng cấu trúc phức tạp hơn hoặc bằng mẹo xoá lười.

Thuật ngữ / câu hỏi	Trả lời
●●● Ba cấu trúc, ba quy tắc lấy ra	Ngăn xếp: mới nhất (LIFO). Hàng đợi: cũ nhất (FIFO). Heap: nhỏ nhất. Đối kho chờ trong cùng một khung thì được duyệt sâu, duyệt rộng, Dijkstra.
●●● Ngăn xếp đơn điệu — ý tưởng	Giữ các vị trí chưa tìm được đáp án theo thứ tự đơn điệu; phần tử mới giải quyết và loại bỏ mọi vị trí nhỏ hơn nó.
●●● Ngăn xếp đơn điệu — vì sao $O(n)$	Mỗi phần tử vào một lần, ra tối đa một lần; tổng $\leq 2n$ thao tác. Khẩu hao kiểu tổng gộp.
●●● Dấu hiệu dùng ngăn xếp đơn điệu	“Với mỗi phần tử, tìm phần tử gần nhất về một phía thoả quan hệ so sánh” — gồm cả biểu đồ cột, nước mưa, số đoạn mà phần tử là cực đại.
●●● Tính chất heap	Mỗi nút $\leq$ các con; lưu bằng mảng, con của i là $2i, 2i+1$. Không đảm bảo gì về quan hệ giữa hai nhánh; mảng không sắp xếp.
●●● Chi phí heap	Lấy nhỏ nhất $O(1)$; chèn và xoá gốc $O(\log n)$; xây từ n phần tử có sẵn $O(n)$.
●●● Vì sao xây heap là $O(n)$	Xây từ dưới lên: nửa số nút ở đáy không phải làm gì; tổng $\sum_h (n/2^h)h$ hội tụ. Cấp số nhân giảm bị chặn bởi hằng số ($A/05$).
●● Cực đại trên cửa sổ trượt	Deque chứa chỉ số có giá trị giảm dần; bỏ đầu khi rơi khỏi cửa sổ, bỏ cuối khi nhỏ hơn phần tử mới. $O(n)$ tổng cộng.
●● Ba mẫu dùng heap	k lớn nhất bằng heap kích thước k ($O(n \log k)$, bộ nhớ $O(k)$); trộn k danh sách đã sắp; trung vị luồng bằng hai heap ngược nhau.
●● Heap không hỗ trợ gì	Xoá phần tử bất kỳ và giảm khoá — cần heap có đánh dấu vị trí, hoặc dùng mẹo xoá lười (kiểm tra bản ghi lỗi thời khi lấy ra).
●● Vì sao Dijkstra cần trọng số không âm	Vì nó chốt đỉnh ngay khi lấy ra khỏi hàng đợi ưu tiên; niềm tin “không có đường rẻ hơn tới sau” chỉ đúng khi mọi cạnh làm chi phí tăng.
●● Heap so với mảng đã sắp	Mảng đã sắp: dữ liệu tĩnh, cần truy vấn theo hạng. Heap: có chèn liên tục, chỉ cần cực trị. Cần cả hai thì dùng cây cân bằng ($B/09$).
● Fibonacci heap	Giảm khoá $O(1)$ khẩu hao, cho Dijkstra $O(m + n \log n)$; hằng số lớn nên hiếm khi thắng trong thực tế.
●● Fibonacci heap có đáng dùng không	Cận $O(m + n \log n)$ cho Dijkstra là thật, nhưng đo thực nghiệm cho thấy pairing heap thắng nó trên mọi tải và thời gian chạy bám theo tỉ lệ trượt cache L1. Thi đấu: heap nhị phân kèm xoá lười.

●●● phải thuộc ●● nên nắm ● biết là có

I Câu hỏi ôn tập có đáp án (che phần đáp án, tự trả lời trước)

1. Khi nào tôi nên viết lại đệ quy thành vòng lặp với ngăn xếp tường minh?

| Chủ yếu khi độ sâu đệ quy có thể lớn — duyệt sâu trên đồ thị 10^5 đỉnh dạng đường thẳng sẽ làm tràn ngăn xếp trong nhiều môi trường. Lý do phụ nhưng cũng quan trọng: khi bạn cần lưu trạng thái tùy biến ở mỗi mức, hoặc cần tạm dừng và tiếp tục việc duyệt. Cái giá là code dài hơn và dễ sai hơn, nên đừng làm khi không cần: đệ quy dễ đọc hơn và khớp với chứng minh quy nạp ($A/04$).

2. Ngăn xếp đơn điệu tăng hay giảm — làm sao khỏi nhầm?

| Đừng nhớ theo tên, hãy suy từ mục tiêu. Nếu bạn tìm “phần tử lớn hơn kế tiếp”, thì những phần tử còn chờ đáp án phải là những phần tử chưa gặp ai lớn hơn — và nếu $a_i \leq a_j$ với $i < j$ thì a_i không thể còn chờ khi a_j đã vào, nên ngăn xếp giữ giá trị giảm dần từ đáy lên đỉnh. Cách kiểm tra nhanh: viết mảng bốn phần tử ra giấy và chạy tay — nhanh hơn nhớ quy tắc.

3. Tôi dùng hàng đợi ưu tiên cho Dijkstra và bị quá thời gian. Nguyên nhân hay gặp?

| Ba nguyên nhân, theo thứ tự tần suất. Một, quên bỏ qua bản ghi lỗi thời khi lấy ra — không có dòng kiểm tra đó thì mỗi đỉnh bị xử lý nhiều lần và số cạnh duyệt tăng vọt. Hai, đẩy vào hàng đợi mọi cạnh kể cả khi không cải thiện được khoảng cách. Ba, dùng cấu trúc nặng làm khoá (chuỗi, cặp lớn) khiến hàng số so sánh cao. Kiểm điểm một trước vì nó là lỗi phổ biến nhất.

4. Deque và hai heap — khi nào dùng cái nào cho bài cửa sổ trượt?

| Nếu bạn chỉ cần cực trị của cửa sổ, deque đơn điệu cho $O(n)$ và hằng số rất nhỏ — luôn chọn nó. Nếu bạn cần trung vị hoặc phần tử thứ k của cửa sổ thì deque không làm được, và bạn cần hai heap có xoá lười hoặc một cây cân bằng (B/09), chấp nhận $O(n \log n)$. Nguyên tắc chung: dùng cấu trúc yếu nhất còn trả lời được câu hỏi của bạn.

5. Vì sao heapsort ít được dùng dù có đảm bảo $O(n \log n)$ ở trường hợp xấu nhất?

| Vì nó truy cập bộ nhớ theo kiểu nhảy cóc giữa các tầng của cây, rất không thân thiện với cache, trong khi quicksort quét gần như tuần tự. Kết quả là quicksort thường nhanh hơn đáng kể trên máy thật dù tệ hơn trên giấy. Giải pháp thực tế của các thư viện là kết hợp: bắt đầu bằng quicksort, chuyển sang heapsort khi độ sâu đệ quy vượt ngưỡng — đúng ý tưởng introsort[3], một ví dụ đẹp cho chương E/26.

6. Có cấu trúc nào thay heap được không?

| Có vài lựa chọn tùy bối cảnh. Nếu khoá là số nguyên nhỏ và tăng dần theo thời gian — ví dụ Dijkstra với trọng số nhỏ — thì hàng đợi theo nhóm (bucket queue) cho $O(1)$ mỗi thao tác. Nếu chỉ có hai loại trọng số 0 và 1, deque thay được hoàn toàn và cho thuật toán 0–1 BFS (C/17). Nếu cần hợp nhất hai hàng đợi thường xuyên, các cấu trúc heap có thể hợp nhất như pairing heap sẽ hợp hơn. Câu hỏi luôn là: cấu trúc yếu nhất còn đủ dùng ở đây là gì.

7. Trong hệ thống thật, hàng đợi ưu tiên có nhược điểm gì mà bài tập không nhắc?

| Có, và nó quan trọng: sự bỏ đói. Nếu luôn phục vụ phần tử ưu tiên cao nhất, các phần tử ưu tiên thấp có thể chờ vô hạn. Hệ điều hành và bộ lập lịch tác vụ giải quyết bằng cách tăng dần độ ưu tiên theo thời gian chờ, tức là biến khoá thành hàm của cả nội dung lẫn thời gian. Đây là ví dụ điển hình của việc bài toán thật có thêm ràng buộc công bằng mà mô hình thuật toán thuần túy bỏ qua — chủ đề của E/26.

| Bài tập (mức: ● khởi động ●● chuẩn ●●● khó)

Bài	Nguồn	Điều cần nhận ra
● Kiểm tra dãy ngoặc hợp lệ	LeetCode “Valid Parentheses”	Ứng dụng ngăn xếp cổ điển nhất; chú ý trường hợp còn dư ngoặc mở khi hết chuỗi.
● Trộn k danh sách đã sắp	LeetCode “Merge k Sorted Lists”	Heap kích thước k ; so sánh với cách trộn đôi một để thấy $O(N \log k)$ từ đâu ra.
● k phần tử lớn nhất trong luồng	LeetCode “Kth Largest Element in a Stream”	Heap nhỏ nhất kích thước k ; giải thích vì sao không dùng heap lớn nhất kích thước n .
●● Phần tử lớn hơn kế tiếp cho mọi vị trí	CSES “Nearest Smaller Values”[4]	Ngăn xếp đơn điệu; viết bất biến ra trước khi cài.
●● Cực đại trên mọi cửa sổ độ dài k	CSES “Sliding Window Cost”-loại; LeetCode “Sliding Window Maximum”	Deque đơn điệu; chú ý lưu chỉ số chứ không lưu giá trị.
●● Trung vị của luồng dữ liệu	LeetCode “Find Median from Data Stream”	Hai heap cân bằng; bất biến về chênh lệch kích thước là chỗ dễ sai nhất.
●● Nhiệm vụ và hạn chót, tối đa số việc làm được	Bài tập kinh điển	Tham lam kết hợp heap: nhận hết rồi bỏ việc tệ nhất khi quá tải — mẫu “tham lam có hối tiếc”.
●●● Hình chữ nhật lớn nhất trong biểu đồ cột	LeetCode “Largest Rectangle in Histogram”	Ngăn xếp đơn điệu; mẫu chót là hiểu rằng lúc một cột bị lấy ra thì cả hai biên của nó đã xác định.

Bài	Nguồn	Điều cần nhận ra (tiếp)
●●● Hình chữ nhật toàn số 1 lớn nhất trong ma trận nhị phân	LeetCode “Maximal Rectangle”	Quy về bài biểu đồ cột trên từng hàng — ví dụ đẹp về việc quy bài lạ về bài đã giải.
●●● Nước mưa mắc kẹt bằng ngăn xếp	LeetCode “Trapping Rain Water”	Đã gặp ở B/06 bằng hai con trỏ; lần này bắt buộc dùng ngăn xếp để thấy hai góc nhìn.
●●● Tổng của cực tiểu trên mọi đoạn con	LeetCode “Sum of Subarray Minimums”	Với mỗi phần tử, đếm số đoạn mà nó là cực tiểu bằng ngăn xếp đơn điệu; cẩn thận với phần tử bằng nhau để khỏi đếm trùng.
●●● Sắp xếp ngoài: trộn nhiều tệp đã sắp lớn hơn bộ nhớ	Bài tập hệ thống	Heap k đường; nối trực tiếp với mô hình bộ nhớ ngoài ở D/24.

Kết chương và đọc tiếp

Ba cấu trúc, ba quy tắc chọn phần tử tiếp theo, và một bài học chung đáng mang theo cả phần B: hãy dùng cấu trúc *yếu nhất mà vẫn trả lời được câu hỏi của bạn*. Heap rẻ hơn cây cân bằng vì nó biết ít hơn; deque rẻ hơn heap vì nó biết còn ít hơn nữa. Mỗi lần bạn đòi hỏi thêm một khả năng, bạn trả thêm tiền — và rất nhiều lời giải chậm là do đòi hỏi những khả năng không dùng tới.

Hai chương tiếp theo là hai câu trả lời đối lập cho bài toán tra cứu. Chương B/08 hỏi “làm sao tìm được phần tử theo khoá thật nhanh nếu tôi *không* cần thứ tự”, và câu trả lời là băm — kèm một cam bẫy an ninh mà rất ít giáo trình nói tới. Chương B/09 hỏi câu ngược lại: nếu tôi *cần* thứ tự thì phải trả thêm bao nhiêu.

Đọc thêm: công trình gốc (chỉ liệt kê nguồn đã qua phản biện; bản tiền ấn được ghi rõ)

Mức	Công trình và tác giả	Nơi công bố, năm	Vì sao đọc, đọc phần nào
nền tảng	Fredman & Tarjan, “Fibonacci Heaps and Their Uses in Improved Network Optimization Algorithms”	Journal of the ACM, 1987	Nguồn của cận $O(m + n \log n)$ cho Dijkstra. Đọc mục mô tả cấu trúc và hàm thế năng; phần ứng dụng cho cây khung nhỏ nhất đọc sau cũng được.
nền tảng	Larkin, Sen, Tarjan, “A Back-to-Basics Empirical Study of Priority Queues”	ALENEX (SIAM), 2014	Số đo thật cho mười biến thể heap. Đọc phần kết luận và bảng Dijkstra trên bản đồ đường — đây là tài liệu chữa bệnh “chọn cấu trúc theo bảng tiệm cận”.
tra khi cần	Tarjan, “Amortized Computational Complexity”	SIAM Journal on Algebraic and Discrete Methods, 1985	Đọc khi muốn hiểu phương pháp thế năng đủ kỹ để tự phân tích một cấu trúc hoãn việc.
tra khi cần	Sedgewick & Wayne, <i>Algorithms</i> (4th ed.), chương 2.4	Addison-Wesley, 2011	Bản trình bày heap nhị phân kèm số đo và mã chạy được; hợp để đối chiếu cài đặt của bạn.

Nguồn

- [1] M. L. Fredman and R. E. Tarjan, “Fibonacci Heaps and Their Uses in Improved Network Optimization Algorithms,” *Journal of the ACM*, vol. 34, no. 3, pp. 596–615, 1987.
- [2] D. H. Larkin, S. Sen, and R. E. Tarjan, “A Back-to-Basics Empirical Study of Priority Queues,” in *Proceedings of the 16th Workshop on Algorithm Engineering and Experiments (ALENEX)*, SIAM, 2014.
- [3] D. R. Musser, “Introspective Sorting and Selection Algorithms,” *Software: Practice and Experience*, vol. 27, no. 8, pp. 983–993, 1997.
- [4] A. Laaksonen. “CSES Problem Set.” (2026), address: <https://cses.fi/problemset/> (visited on 09/06/2026).